

|                        |                        |
|------------------------|------------------------|
| <b>Name: Aryan Raj</b> | <b>Class: SYIT</b>     |
| <b>Class: SYIT</b>     | <b>Roll No. : S003</b> |

## Mini Project Report

### 1. Title of Project: - STUDENT PERFORMANCE ANALYSIS AND DASHBOARD

Tools Used: Python, NumPy, Pandas, Matplotlib, Seaborn, Beautiful Soup

### 2. Introduction

Student performance analysis helps educational institutions understand how factors such as study hours and attendance affect academic results. In this project, a student performance dataset containing 120 records is analyzed using Python libraries. Different statistical methods and visualizations are used to identify patterns and present the results through a dashboard.

### 3. Objective

- To analyze student performance data using Python.
- To clean and organize the dataset using Pandas.
- To perform numerical and statistical analysis using NumPy and Pandas.
- To create meaningful visualizations using Matplotlib and Seaborn.
- To collect sample public web data (grading scheme) using Beautiful Soup.
- To present important results using a Student Performance Analysis Dashboard.

### 4. Dataset Description

The student performance dataset contains 120 records and 7 columns (a Total\_Score column is added during cleaning).

| Column      | Description                                |
|-------------|--------------------------------------------|
| Student_ID  | Unique ID of the student                   |
| Study_Hours | Average study hours per day                |
| Attendance  | Attendance percentage                      |
| Math_Score  | Marks obtained in Mathematics (out of 100) |

|               |                                                    |
|---------------|----------------------------------------------------|
| Science_Score | Marks obtained in Science (out of 100)             |
| English_Score | Marks obtained in English (out of 100)             |
| Grade         | Performance grade (Excellent, Good, Average, Fail) |

## 5. Tools and Libraries Used

| Library        | Purpose                                     |
|----------------|---------------------------------------------|
| NumPy          | Numerical calculations and array operations |
| Pandas         | Dataset loading, cleaning and analysis      |
| Matplotlib     | Data visualization                          |
| Seaborn        | Statistical visualization                   |
| Beautiful Soup | Extracting data from HTML/web pages         |

## 6. Data Cleaning

Cleaning operations performed:

1. Removed extra spaces from column names using `str.strip()`.
2. Removed duplicate records using `drop_duplicates()`.
3. Handled missing values in numerical columns using their mean values.
4. Created a new column `Total_Score` by adding the three subject scores.

### Code:-

```
# Data Cleaning
df.columns = df.columns.str.strip()
df = df.drop_duplicates()

for col in ["Study_Hours", "Attendance", "Math_Score",
           "Science_Score", "English_Score"]:
    df[col] = df[col].fillna(df[col].mean())

df["Total_Score"] = (df["Math_Score"] + df["Science_Score"]
                    + df["English_Score"])

print("Data Cleaning Completed!")
print("\nMissing Values After Cleaning:")
print(df.isnull().sum())
print("\nDuplicate Records After Cleaning:")
print(df.duplicated().sum())
```

## Output:-

Data Cleaning Completed!

Missing Values After Cleaning:

```
Student_ID      0
Study_Hours     0
Attendance      0
Math_Score      0
Science_Score   0
English_Score   0
Grade           0
Total_Score     0
dtype: int64
```

Duplicate Records After Cleaning:

0

Shape after cleaning: (120, 8)

## 7. Data Analysis

### Code:-

```
# Data Analysis
avg_math = np.mean(df["Math_Score"])
max_math = np.max(df["Math_Score"])
min_math = np.min(df["Math_Score"])
avg_sci = df["Science_Score"].mean()
avg_eng = df["English_Score"].mean()
avg_hours = df["Study_Hours"].mean()
max_hours = df["Study_Hours"].max()
avg_att = df["Attendance"].mean()
min_att = df["Attendance"].min()
top = df.loc[df["Total_Score"].idxmax()]
corr = df["Study_Hours"].corr(df["Total_Score"])
grade_count = df["Grade"].value_counts()

print("DATA ANALYSIS RESULTS")
print("-----")
print("Average Math Score:", round(avg_math, 2))
print("Maximum Math Score:", max_math)
print("Minimum Math Score:", min_math)
print("Average Science Score:", round(avg_sci, 2))
```

```

print("Average English Score:", round(avg_eng, 2))
print("Average Study Hours:", round(avg_hours, 2), "hrs")
print("Maximum Study Hours:", max_hours, "hrs")
print("Average Attendance:", round(avg_att, 2), "%")
print("Minimum Attendance:", min_att, "%")
print("Top Student:", top["Student_ID"],
      "with Total Score", top["Total_Score"])
print("Correlation (Study Hours vs Total Score):",
      round(corr, 3))
print("\nGrade Count:")
print(grade_count)

```

### Output:-

#### DATA ANALYSIS RESULTS

```

-----
Average Math Score: 60.28
Maximum Math Score: 90.0
Minimum Math Score: 27.0
Average Science Score: 58.93
Average English Score: 59.69
Average Study Hours: 4.95 hrs
Maximum Study Hours: 8.8 hrs
Average Attendance: 74.84 %
Minimum Attendance: 55.0 %
Top Student: S023 with Total Score 251.0
Correlation (Study Hours vs Total Score): 0.912

```

Grade Count:

Grade

Average      46

Good          38

Fail          30

Excellent    6

Name: count, dtype: int64

## 8. Web Data Collection using BeautifulSoup

Beautiful Soup was used to extract the grading scheme from an HTML table.

### Code:-

```

from bs4 import BeautifulSoup

html = """<table id='marks'>
<tr><th>Grade</th><th>Marks Range</th></tr>
<tr><td>Excellent</td><td>80 and above</td></tr>

```

```
<tr><td>Good</td><td>65 - 79</td></tr>
<tr><td>Average</td><td>50 - 64</td></tr>
<tr><td>Fail</td><td>Below 50</td></tr></table>"""
```

```
soup = BeautifulSoup(html, "html.parser")
for row in soup.find_all("tr")[1:]:
    cells = [td.text for td in row.find_all("td")]
    print(cells[0], "->", cells[1])
```

### Output:-

```
Excellent -> 80 and above
Good -> 65 - 79
Average -> 50 - 64
Fail -> Below 50
```

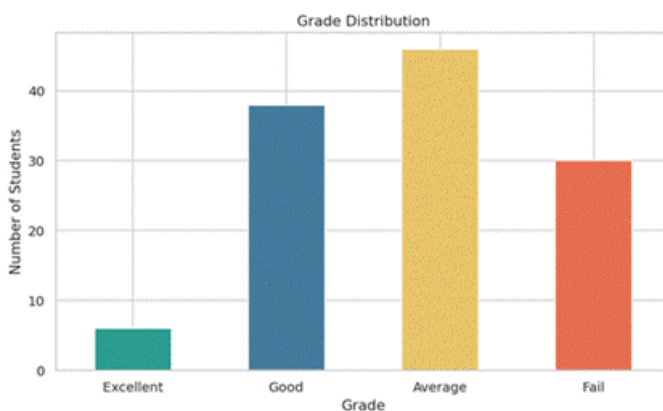
## 9. Data Visualizations

### 1. Bar chart

#### Code:-

```
plt.figure(figsize=(8, 5))
df["Grade"].value_counts().reindex(
    ["Excellent", "Good", "Average", "Fail"]).plot(kind="bar")
plt.title("Grade Distribution")
plt.xlabel("Grade")
plt.ylabel("Number of Students")
plt.xticks(rotation=0)
plt.show()
```

#### Output:-



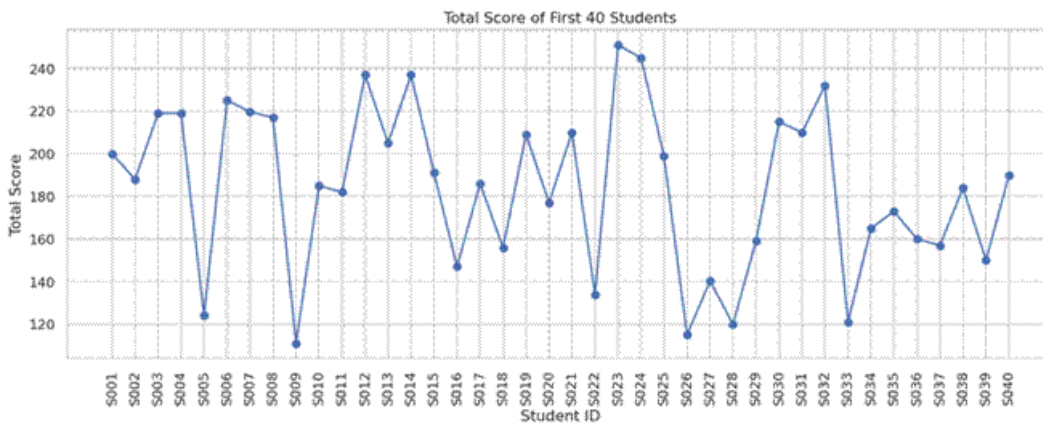
### 2. Line chart

#### Code:-

```
plt.figure(figsize=(12, 5))
```

```
plt.plot(df["Student_ID"].iloc[:40],
         df["Total_Score"].iloc[:40], marker="o")
plt.title("Total Score of First 40 Students")
plt.xlabel("Student ID")
plt.ylabel("Total Score")
plt.xticks(rotation=90)
plt.tight_layout()
plt.show()
```

**Output:-**

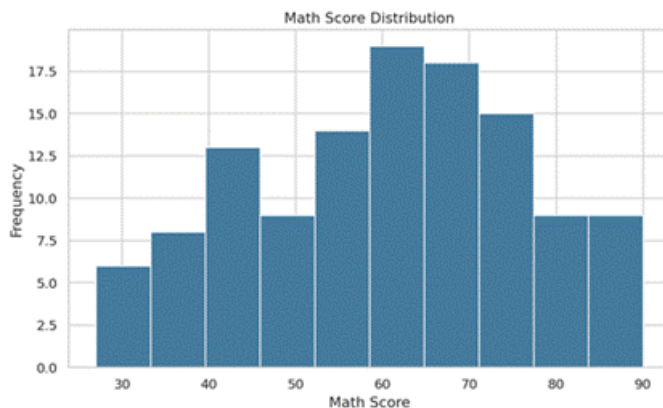


### 3. Histogram

**Code:-**

```
plt.figure(figsize=(8, 5))
plt.hist(df["Math_Score"], bins=10)
plt.title("Math Score Distribution")
plt.xlabel("Math Score")
plt.ylabel("Frequency")
plt.show()
```

**Output:-**

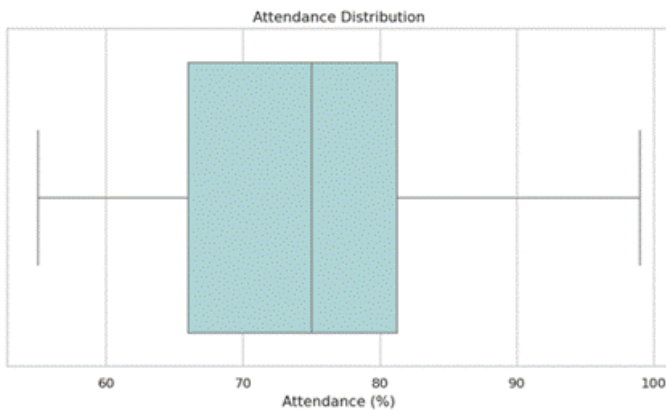


#### 4. Box Plot – Attendance

**Code:-**

```
plt.figure(figsize=(8, 5))
sns.boxplot(x=df["Attendance"])
plt.title("Attendance Distribution")
plt.xlabel("Attendance (%)")
plt.show()
```

**Output:-**

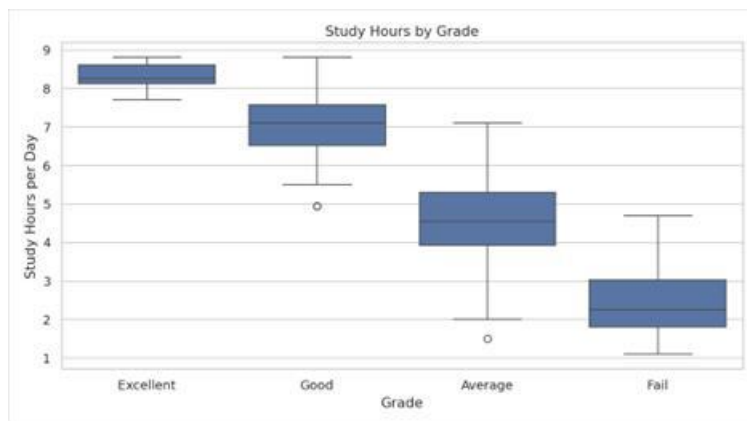


#### 5. Box Plot – Study Hours by Grade

**Code:-**

```
plt.figure(figsize=(9, 5))
sns.boxplot(x="Grade", y="Study_Hours", data=df,
            order=["Excellent", "Good", "Average", "Fail"])
plt.title("Study Hours by Grade")
plt.xlabel("Grade")
plt.ylabel("Study Hours per Day")
plt.show()
```

**Output:-**

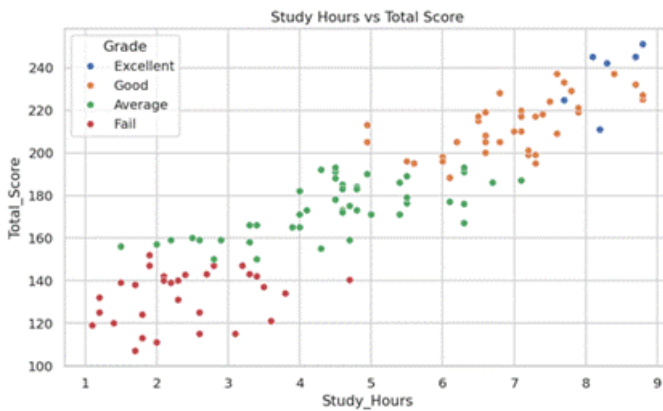


## 6. Scatter Plot

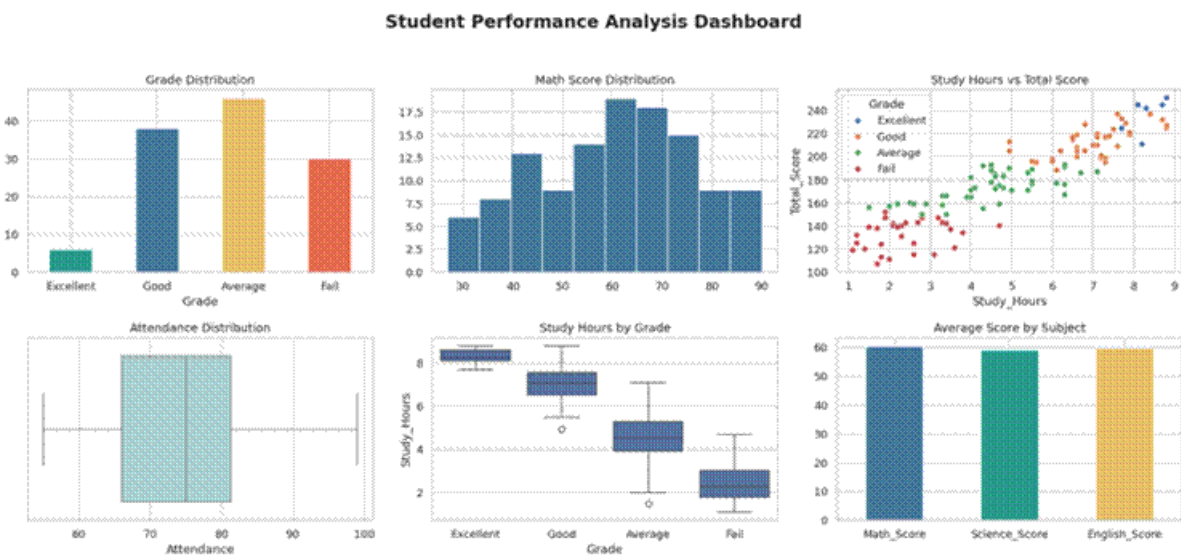
Code:-

```
plt.figure(figsize=(8, 5))
sns.scatterplot(x="Study_Hours", y="Total_Score",
                hue="Grade", data=df)
plt.title("Study Hours vs Total Score")
plt.show()
```

Output:-



## 10. Dashboard



## 11. Key Findings/Observations

- The dataset contains 120 student records after cleaning (4 duplicates removed).
- The dataset contains 4 grade categories: Excellent, Good, Average and Fail.

- The highest Math score is 90 and the lowest is 27; the average is about 60.28.
- Average scores in Science (58.93) and English (59.69) are close to Math.
- The average study time is about 4.95 hours per day and average attendance is about 74.84%.
- Study hours and total score show a strong positive correlation (about 0.912).
- Most students fall in the Average (46) and Good (38) grades; 30 students failed and 6 were Excellent.
- The top performer is student S023 with a total score of 251.

## **12. Conclusion**

The Student Performance Analysis and Dashboard project successfully analyzed student data using Python. Pandas and NumPy were used for data loading, cleaning, and analysis, while Matplotlib and Seaborn were used to create meaningful visualizations. Beautiful Soup was used for basic web data collection from HTML content. The analysis shows that students who study more hours tend to score higher, and the final dashboard presents the important statistics, visualizations, and findings in an easy-to-understand format.